



同济大学软件学院

School of Software Engineering, Tongji University



Software Testing Foundation Level

Chapter 3: Static techniques

刘琴 *Zin Liu*



3 Static techniques

3.1 Static techniques and the test process

3.2 Review process

3.3 Static analysis by tools



3.1 Static Test

- Test object is not provided with test data, not executed
 - Analyzed, by persons and tools
 - Used for all documents relevant to software development and maintenance
 - Tool-supported static analysis is only for documents with a formal structure
- Goal: find defects and deviations, optimizing the development process

3.1.1 Structured Group Evaluations



- Reviews, applying human analytical capabilities
- The only possibility to check the semantics of a document
- Relying on colleagues of the author to provide mutual feedback: peer reviews
- A means for quality assurance
 - Eliminating defects and inconsistencies



3.1.2 Positive Effects

- Cheaper defect elimination
- Shortened development time
- Decreased costs and time for dynamic tests
- Cost reduction during the whole product life
- Reduced failure rate during operation
- Mutual learning
- Helping the author find forgotten issues
- Whole team feeling responsible for the quality



3.1.3 Some Issues

- Potential problem: in a badly moderated review session
 - Author feeling that he (not the document) is subject to critical scrutiny
- Cost of reviews: 10-15% of development budget
- Savings of reviews: 14-25%
- If reviews are systematically used and efficiently run, more than 70% of defects can be found and repaired
- Success factors
 - Every review has a clear goal, formulated beforehand
 - Right people are chosen as review participants



3 Static techniques

3.1 Static techniques and the test process

3.2 Review process

3.3 Static analysis by tools



3.2.1 Types of Reviews

- Informal: no real process (hallway chats, buddy tests, pair programming), yet useful, cheap, popular
- Technical: documented and defined defect removal process, involving technical experts but not managers
- Walkthrough: author “walks through” review item
- Inspection: a trained moderator (other than the author) leads the inspection team (with defined roles) through a formal inspection process (rules, checklists, entry and exit criteria), which includes gathering defect removal metrics
- When walkthroughs, technical reviews or inspections are performed by a peer group, the review may be called a peer review



3.2.1.1 Walkthrough

- Manual, informal
- Less focus on preparation
- Typical usage situations (scenarios) are discussed
- Test cases may be played through
- Reviewers try to find possible errors/defects by spontaneously asking questions
- Useful for small teams (up to 5 persons), for checking noncritical documents
- Objective: mutual learning, development of understanding of review object, error detection



3.2.1.2 Inspection

- The most formal review
- Reviews use checklists with criteria for checking different aspects
- Goal: finding unclear items and possible defects, measuring quality, improving quality
- Called design inspection or code/software inspection
- Inspection meeting
- Pair Programming, two programmers work together at one workstation ?
- In addition: optimizing the development process



3.2.1.3 Technical Review

- Focus: compliance of the document with the specification, fitness for its intended purpose, and compliance to standards
- Technical experts as reviewers (some should not be project participants for avoiding project blindness)
- High preparation effort



3.2.1.4 Informal Review

- A light version of review
- Initiated by the author in most cases
- No meeting: just a simple author-reader-cycle
 - Cross-reading by colleagues
- Results need not be explicitly documented
 - A list of remarks or the revised document is enough
- Types of informal review
 - Pair programming, buddy testing, code swapping
- Very common and highly accepted due to minimal effort required



3.2.1.5 Pair Programming

- An agile software development technique
- Two programmers work together at one workstation
 - Driver: writing the source code
 - Observer/navigator: reviewing each line of code
 - The two programmers switch roles frequently
- Advantages
 - Economics
 - Design quality
 - Satisfaction
 - Learning
 - Team-building and communication
- Pairing variations
 - Expert-expert, expert-novice, novice-novice

3.2.1.5 Pair Programming (cont.)



- Remote pair programming (virtual pair programming, distributed pair programming)
 - Two programmers are in different locations
 - Working via a collaborative real-time editor, shared desktop, or a remote pair programming IDE plugin

3.2.2 Consensus and Understanding

- Incompleteness and ambiguity can hide the real meaning of the specifications
- Agreement and uniform understanding of the specifications

Long before any code exists, the specification must be handed to an outside testing group to be scrutinized for completeness and clarity. As [V.A.] Vyssotsky [of Bell Lab's Safeguard Project] says, the developers themselves cannot do this: "They won't tell you they don't understand it; they will happily invent their way through the gaps and obscurities."

*– Fred Brooks
The Mythical Man-Month
1975*





3.2.3 A Generic Review Process

1. Planning

2. Kick-off

3. Preparation

4. Review meeting

5. Rework/repair

6. Follow-up

Includes estimating and planning, training participants, etc.

These steps of the process repeat per each item reviewed. Preparation is usually one to two hours alone. Meeting is one to two hours together. Rework/repair is fixing the bugs found.

The details of the review process depend on the specific review type used on the project

Follow-up includes on individual items as well as overall process improvement analysis, evaluation of defect (bug) removal at phase exit reviews (exit meetings), etc.



3.2.4 Roles and Responsibilities

- Moderator: Lead the review meetings
- Scribe or secretary: Gather information on findings
- Author: Describe, explain, answer questions on item
- Reviewer/inspector: Find defects (bugs) in item
- Manager: Plan, arrange resources and training, support, analyze process metrics
- In some cases, one person may play multiple roles
 - Authors sometimes act as moderators
 - One of the reviewers can act as the secretary
 - The specifics are determined by the type of review



3.2.5 Suggestions for Successful Reviews

- Provide training
- Review the product, not the producer
- Set and follow agenda and objectives
- Limit debate
- Focusing on finding, not fixing, problems
- Take written notes
- Limit and carefully select participants
- Insist on preparation (e.g., by having people submit notes)
- Develop a checklist for each type of item that is reviewed
- Review the reviews
- Use the right techniques
- Ensure management support
- Learn and get better!

3.2.6 Common Requirements and Design Bugs



- Ambiguities: What exactly does that mean?
 - E.g.: System shall allow user to read ISP e-mail
 - What ISPs? What size e-mails? Attachments?
- Incompleteness: Okay, and then what?
 - E.g.: Upon three invalid passwords, system shall lock user's account...
 - For how long? How to unlock? Who can unlock?
- Untestability: How can I check this item?
 - E.g.: System shall provide 100% availability
 - No known test technique to demonstrate perfect availability
- Excessive dependencies, coupling and complexity
 - Look for ugly design diagrams and confusing requirements

3.2.7 IEEE 1028 Software Review Standard



1. Overview
 - Purpose, scope, conformance, organization, application
2. References
3. Definitions
4. Management reviews
 - Responsibilities, inputs/outputs, entry/exit criteria, procedures
5. Technical reviews
 - Responsibilities, inputs/outputs, entry/exit criteria, procedures

3.2.7 IEEE 1028 Software Review Standard



6. Inspections

- Responsibilities, inputs/outputs, entry/exit criteria, procedures, data collection, process improvement

7. Walkthroughs

- Responsibilities, inputs/outputs, entry/exit criteria, procedures, data collection, process improvement

8. Audits

- Responsibilities, inputs/outputs, entry/exit criteria, procedures



3.2.8 Selection Criteria

- Depending on how thorough the review needs to be, and effort that can be spent
 - The form of review results (documentation, or presented informally)
 - Date and time for the review
 - Technical knowledge from different disciplines or not
 - Level of technical knowledge required
 - Presentation effort appropriate or not
 - Is review object formally written
 - How much management support



3.2.9 Success Factors

- Reviews help improve the examined documents
- Human and psychological factors have a strong influence
- Testers should be used as reviewers
- Considering type and level of examined document, and state of knowledge of participants
- Checklists and guidelines should be used
- Training
- Management can support
- Continuous learning



3 Static techniques

3.1 Static techniques and the test process

3.2 Review process

3.3 Static analysis by tools



3.3 Static Analysis

- Objective similar to reviews, but tools do the analysis
 - Example: spell check is a form of static analyzer
- Document to be analyzed must follow a formal structure, in order to be checked by a tool
 - Technical requirements
 - Software architecture
 - Software design (e.g. UML diagrams)
 - HTML/XML documents
 - Formal models
 - Program code



3.3 Static Analysis (cont.)

- Analysis tools often produce a long list of warnings and comments
 - Information must be handled intelligently
- Static analysis and reviews are closely related
 - Static analysis should be performed before review, if documents are formal enough
- Not all defects can be found using static testing
- Some inconsistencies and defect-prone areas are difficult to find by dynamic testing
 - Detecting violation of programming standards or use of forbidden error-prone program constructs is only possible with static analysis (or reviews)



3.3 Static Analysis (cont.)

- Defects and dangerous constructions that can be detected by static analysis
 - Syntax violations
 - Deviations from conventions and standards
 - Control flow anomalies
 - Data flow anomalies
- In addition, static analysis can detect security problems
 - E.g., lack of buffer flow protection, failing to check whether input data out of bounds

3.3.1 Compiler as a Static Analysis Tool



- Violation of syntax can be detected by the compiler, and reported as fault or warning
- Additional information/checks
 - Generating a cross-reference list
 - Checking for correct data type usage
 - Detecting undeclared variables
 - Detecting dead code
 - Detecting overflow/underflow of field boundaries
 - Checking interface consistency
 - Detecting the use of all labels



3.3.2 Conventions and Standards

- Compliance to conventions and standards can be checked with tools
- Only guidelines that can be verified by tools should be accepted in a project
- An additional advantage
 - If the programmers know that the program code is checked for compliance to the programming guidelines, their willingness to work according to the guidelines is much higher



3.3.3 Data Flow Analysis

- Checking the usage of data on paths through the program code
 - An anomaly is an inconsistency that can lead to failure (but does not necessarily do so)
 - May be flagged as a risk
- The analysis checks usage of every variable
 - **Defined (d)**: the variable is assigned a value
 - **Referenced (r)**: the value of the variable is read and/or used
 - **Undefined (u)**: the variable has no defined value



3.3.4 Data Flow Analysis (cont.)

- Data flow anomalies
 - **ur-anomaly**: an undefined value (u) of a variable is read on a program path (r)
 - **du-anomaly**: the variable is assigned a value (d) that becomes invalid/undefined (u) without having been used in the meantime
 - **dd-anomaly**: the variable receives a value for the second time (d) and the first value had not been used (d)



3.3.5 Data Flow Analysis (cont.)

- Example of data flow anomalies

```
void exchange (int& Min, int& Max) {  
    int Help;  
    if (Min > Max) {  
        Max = Help;  
        Max = Min;  
        Help = Min;  
    }  
}
```

- ur-anomaly of the variable *Help*
- dd-anomaly of the variable *Max*
- du-anomaly of the variable *Help*

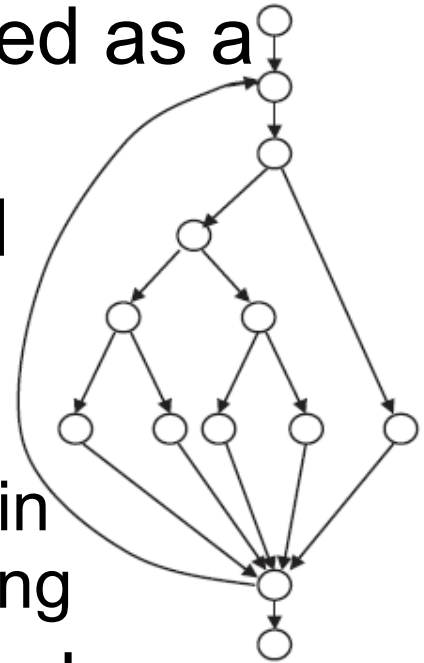


3.3.5 Data Flow Analysis (cont.)

- Not every anomaly leads directly to an incorrect behavior (e.g. du-anomaly)
- An exact examination of the program parts is worthwhile, and further inconsistencies can be discovered

3.3.6 Control Flow Analysis

- Program structure can be represented as a control flow graph
- Possible anomalies can be detected
 - Jumps out of a loop body
 - A structure with several exits
 - Not necessarily lead to failure, but not in accordance with structured programming
- If parts of graph are very complex and relations are not understandable, the program should be revised



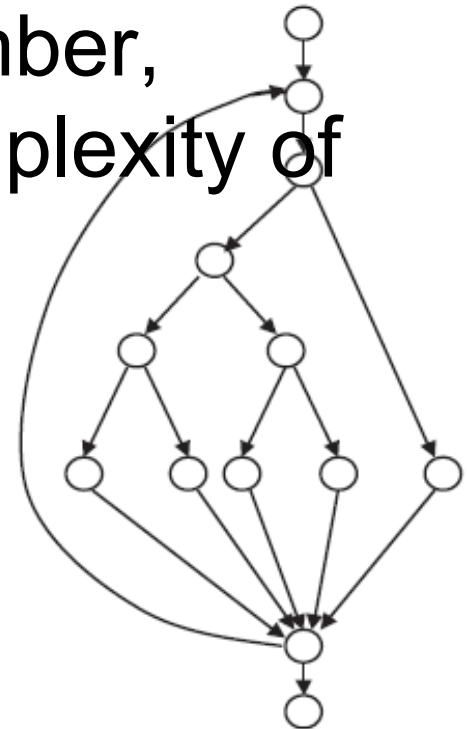
3.3.6 Control Flow Analysis (cont.)



- Predecessor-successor table
 - Showing how every statement is related to others
- If a statement has no predecessor
 - Unreachable (dead code)
- The only exceptions: first/last statements

3.3.7 Determining Metrics

- Static analysis tools also provide measurement values (metrics), for measuring quality characteristics
- Example: the *cyclomatic* number, measuring the structural complexity of code
 - Definition: $v(G) = e - n + 2$
 - Testability and maintainability



Example: Property Graphs

源代码静态分析方法--代码属性图Code Property Graphs

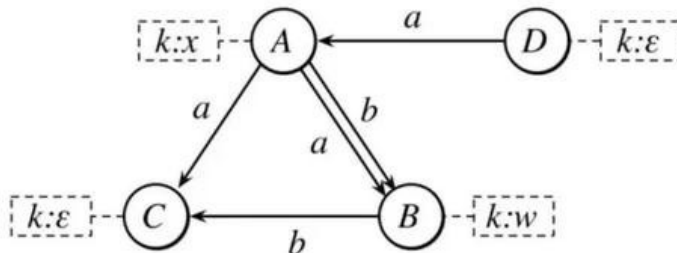
代码属性图是一种针对源代码的语义分析方法，其中涵盖了语法、程序控制流、程序控制依赖以及数据依赖的语义，能够用于发现源代码中的缓冲区溢出、整数溢出、格式化字符串漏洞以及内存泄露漏洞。

代码属性图是将抽象语法树AST、控制流图CFG以及程序依赖图PDG融合为一种联合结构，然后利用图遍历，结合事先定义的漏洞检测规则进行漏洞检测。

注：由于公式无法正常显示，故将PDF版本上传到附件处，需要自取

属性图 Property Graphs

属性图定义为 $G = (V, E, \lambda, \mu)$ ，是有方向的，边带有标签，多重属性图。其中 V 代码node集， E 代表有向边的集合， $\lambda: E \rightarrow \Sigma$ 表示对边进行标注的函数，其中 Σ 是可见字符集， $\mu: (V \cup E) \times K \rightarrow S$ 表示对边和节点进行属性赋值的函数，其中 K 代表属性键的集合， S 代表属性值的集合。



如上图所示属性图，带有四个nodes，每个node都有属性键 $k \in K$ ，但是只有A,B存储了来自属性值集合 $S = x, w$ 中的属性值。

沿着属性图进行信息查询称作图遍历graph traversals，定义为 $T: \rho(V) \rightarrow \rho(V)$ ，表示根据属性图将一组nodes映射到另一组nodes。我们还可将两个图遍历进行联合，比如 $T_0 \circ T_1$ 。用如下可表示一个简单的过滤图遍历 $FILTER_p(X) = \{v \in X : p(v)\}$ ，表示返回节点集合 X 中所有符合预测 $p(V)$ 的节点。

抽象语法树

用下图中代码为例

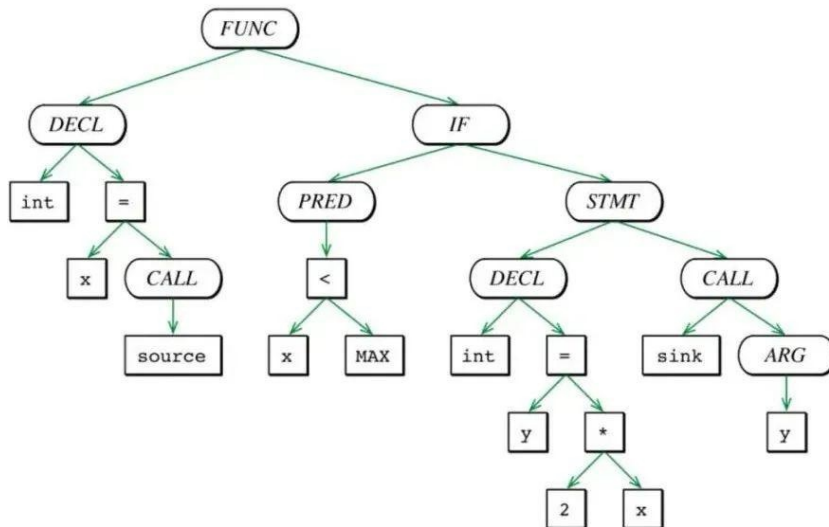
```

void foo()
{
    int x = source();
    if (x < MAX)
    {
        int y = 2 * x;
        sink(y);
    }
}

```

1
2
3
4
5
6
7
8
9

生成的抽象语法树如下



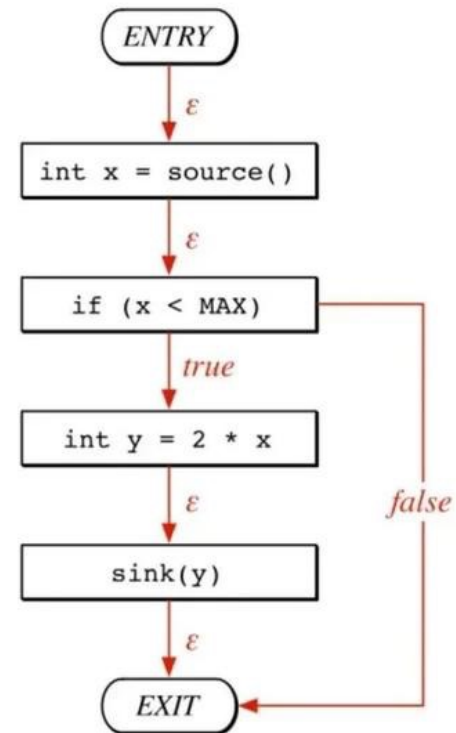
AST能够清晰地展示statements以及expressions之间的嵌套关系，其内部节点是操作符operators（比如加或者减），叶子节点是操作数（比如常量constants或者其他变量identifiers）。

AST可被用来对代码进行转换，进行相似代码比对，但是却无法对代码进行更复杂的检查，因为其中不包含控制流或者数据流关系，比如无法检查dead code或者uninitialized variables。

用属性图表示为： $G_A = (V_A, E_A, \lambda_A, \mu_A)$ ，其中 V_A 表示树节点， E_A 表示为被边标记函数 λ_A 进行标记的边，使 μ_A 为每个节点分配属性值，比如操作数或者操作码。并且为每个node分配了一个顺序，表达其在AST中的结构，因此 $K_A = \{code, order\}$ 。

控制流图CFG

上图例子中生成的CFG如下



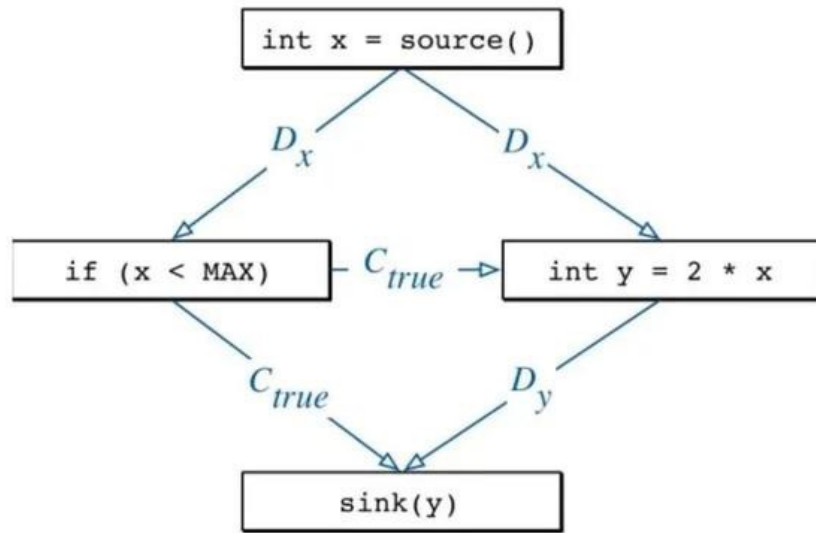
CFG清晰地展示代码中语句的执行顺序，node表示程序语句，其中的边表示跳转关系，分别为true,false, ϵ 。

CFG目前能够应用于识别程序的安全上写文，比如检测一直恶意应用或者引导fuzzing，但是由于其不涉及数据依赖信息，所以其不能够发现涉及数据依赖的漏洞信息。

用属性图表示为 $G_C = (V_C, E_C, \lambda_C, \cdot)$ ，其中 V_C 可用AST内的statement和predicates表示，其中 λ_C 的取值为 $\{true, false, \epsilon\}$ 。

程序依赖图PDG

上图例子中生成的PDG如下



程序依赖图是用于展示程序内部的数据以及控制流的依赖关系，比如可以检查某代码位置的数据能够被哪些数据影响，此位置的代码能够从那些代码位置可达。如上图所示，其中D表示Data，C代表Control，箭头方向代表可达方向。在此图中不能看清楚语句之间的执行顺序，但是其中的依赖关系是清晰可见的。

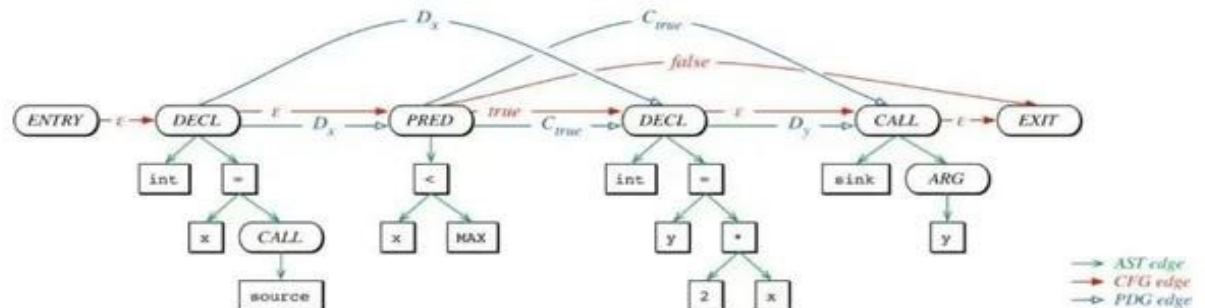
用属性图表示为 $G_P = (V_P, E_P, \lambda_P, \mu_P)$ ，其中 λ_P 的取值为 $\{C, D\}$ ，C表示control dependence，D表示data dependence。并且对数据依赖赋予符号属性比如 x, y ，对控制依赖赋予true或者false。

代码属性图CPG

因此，将上述AST, CFG, PDG融合，就能够将语法、控制流、控制依赖、数据依赖用一个统一的数据结构表示，表示为 $G = (V, E, \lambda, \mu)$ ，其中

1. $V = V_A$
2. $E = E_A \cup E_C \cup E_P$
3. $\lambda = \lambda_A \cup \lambda_C \cup \lambda_P$
4. $\mu = \mu_A \cup \mu_P$

上图例子中生成的CPG如下

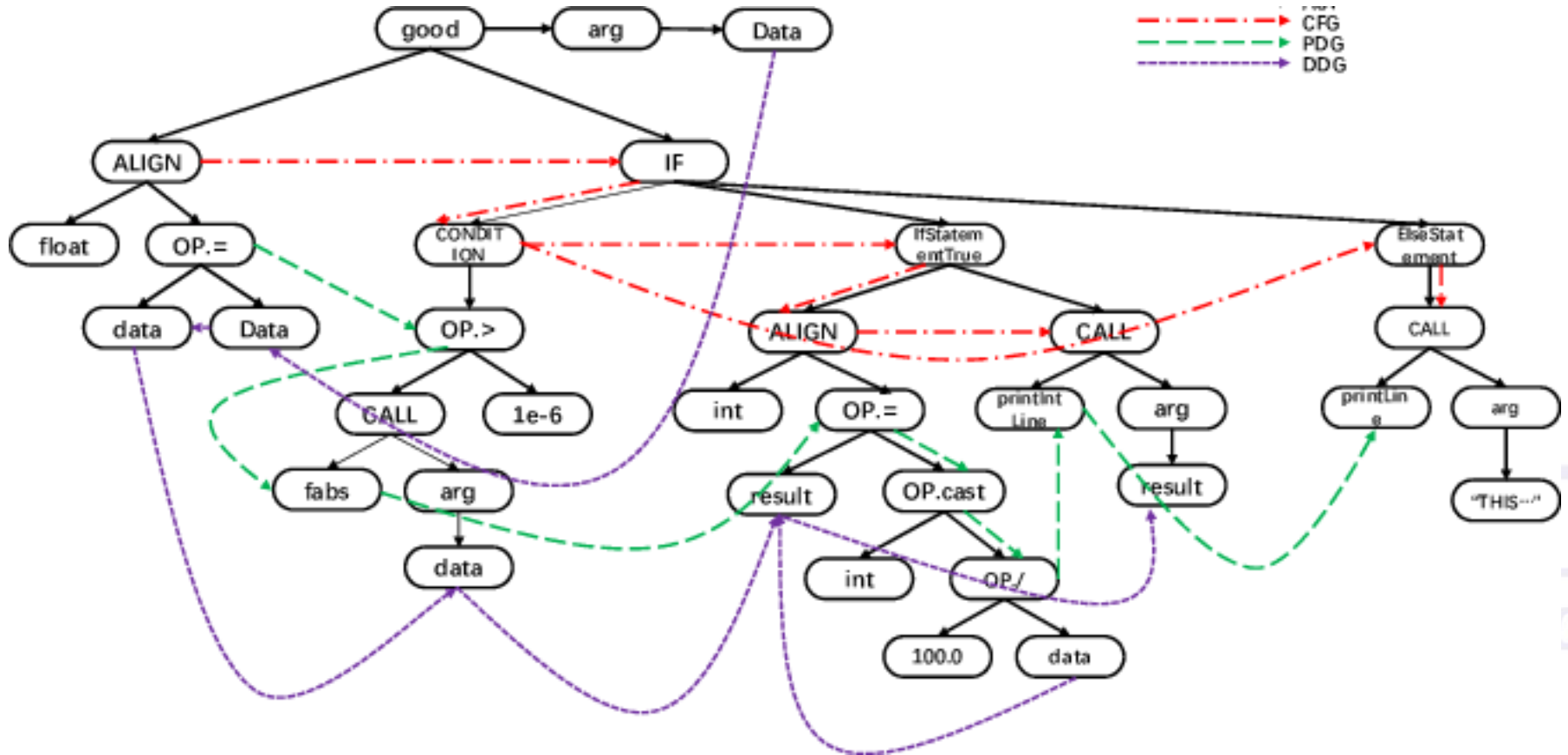


检测已知漏洞

现在已知代码属性图包含程序语法、控制流、控制依赖、数据依赖的信息，并且依据图遍历方法能够对代码进行查询，那么即可对一些已知漏洞类型进行检测。

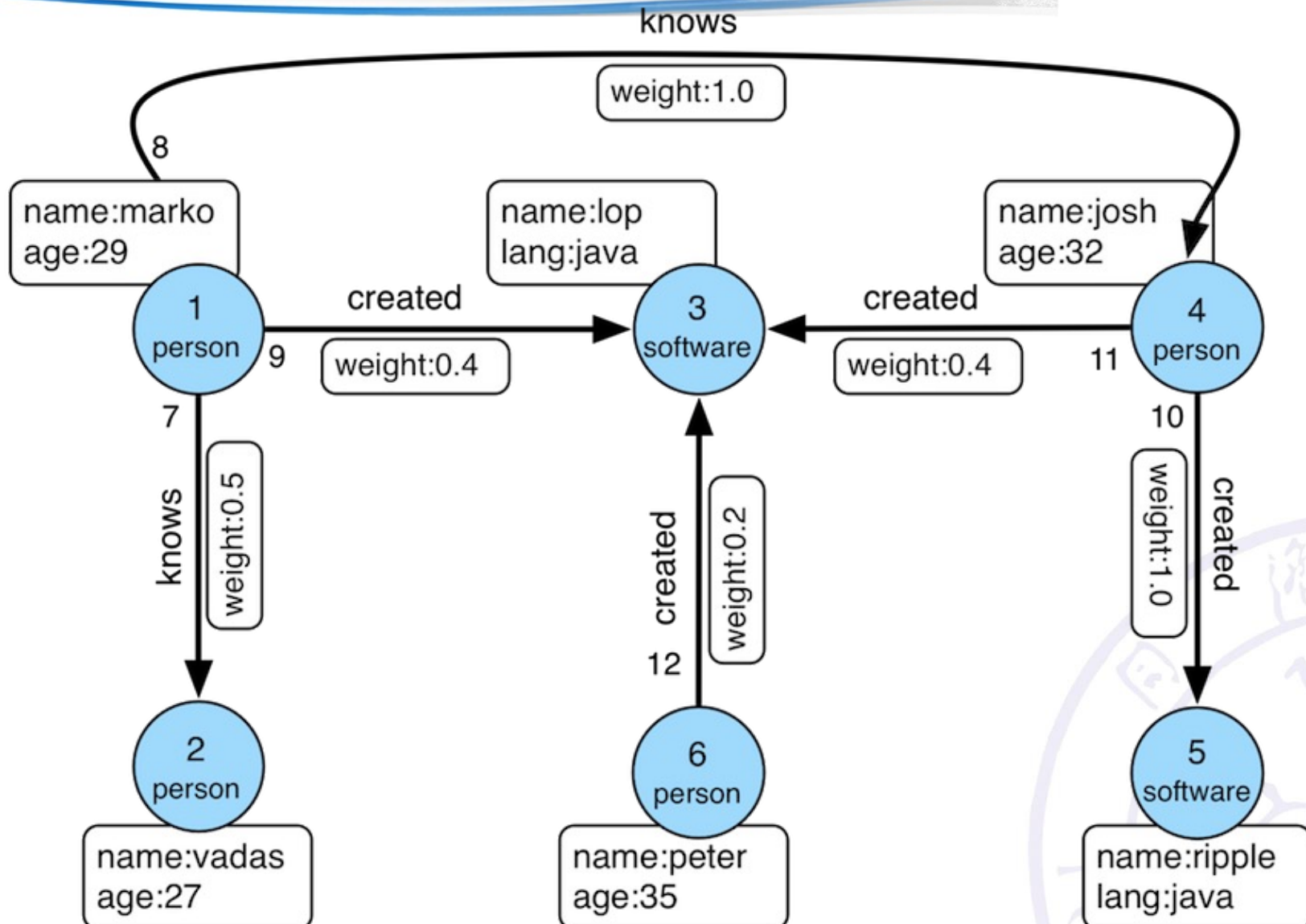
漏洞检测：语法

Code Property Graphs



Program Dependency Graph (PDG) overlaid on top of an **Abstract Syntax Tree (AST)**.

the "Modern" graph





上海市嘉定区曹安公路4800号，同济大学嘉定校区软件学院